TP3

Ejercicio 1.

a.Agregar este programa al shell de Xinu (recuerde cambiar el nombre de main por el nombre que seleccione para el programa)

```
#include <xinu.h>

void produce(void), consume (void);

int32 n = 0;
sid32 semp, semc;

void ejercicio1tp3(void) {
semp = semcreate(1);
semc = semcreate(0);

resume(create(consume,1024,20,"cons",0));
resume(create(produce,1024,20,"prod",0));

}


void produce(void) {

int32 i;

for(i = 1; i <= 2000; i++){
wait(semp);
printf("EL productor produce");
n++;
signal(semc);
}
}


void consume(void) {

int32 i;
for(i = 1; i <= 2000; i++){
wait(semc);
printf("EL valor de n es %d ", n);
signal(semp);
```

```
}
}
```

b. Este programa es un típico programa compuesto de 3 procesos, main, productor, consumidor. Los procesos productor y consumidor se “comunican” a través de una variable compartida. El productor genera valores,y el consumidor los muestra en pantalla. Ejecutar el programa. Describir qué sucede. ¿El programa funciona como el programador pretendió?

El programa sin ninguna modificación, no funciona como el programador pretende, ya que existen dos zonas críticas en el módulo "consume" y en modulo "produce" ya que modifican la misma variable.

c. Modifique el programa utilizando algún mecanismo provisto por el sistema operativo para la sincronización de procesos, de manera que el programa funcione como el programador pretendía.

(En el código anterior ya está modificado)

Ejercicio 2. Ejecutar el programa. ¿Por qué este programa (sus procesos) emiten mensajes de error? ¿Qué es posible hacer para solucionar el problema?

```
#include <xinu.h>
void mutex_init();
int mutex_lock(), mutex_unlock();
int32 pid = -1;


sid32 semaforo;


void mutex_init(){ //En caso de que la creación de error, el mismo crear
del semáforo devuelve error.


semaforo = semcreate(1);
```

```
}

void mutex_lock(){

wait(semaforo);

pid = getpid();

}


int mutex_unlock(){

if(pid == getpid()){ //Verificamos que el mismo proceso sea el que quiera desbloquearlo

signal(semaforo);

pid = -1; //Reiniciamos el pid

return 1;

}else{
return -1; //El mismo proceso intenta hacer lock dos veces.
}

}

}
```

```
#include <xinu.h>

sid32 semmutex;
int32 actualprocess;

void mutex_init(){
    semmutex = semcreate(1);
}

void mutex_lock(){
```

```
    wait(semmutex);
    actualprocess = getpid();
}

int mutex_unlock(){
    int rvalue;
    int32 callingprocess = getpid();
    if(callingprocess == actualprocess){
        signal(semmutex);
        rvalue = 1;
        actualprocess = 0;
    }else{
        rvalue = -1;
    }
    return rvalue;
}
```

```
#include <xinu.h>


void operar(void), incrementar(void);
unsigned char x =0 ;
int result;


void ejercicio2tp3(void) {

mutex_init();


resume(create(operar,1024,20,"process 1",0));
resume(create(incrementar,1024,20,"process 2",0));


sleep(10);
}



void operar(void) {

int y =0;

printf("SI no existen mensajes de ERROR entonces todo va OK \n");
while(1){
/*si x es multiplo de 10*/
```

```
mutex_lock();
if((x%10) == 0) {
y = x*2;

/*si no es multiplo de 10 entonces hubo un error*/

if((y%10) != 0) {
printf("\r ERROR!! y = %d, x = %d \r",y,x); }

}


mutex_unlock();

}

}



void incrementar(void){
while(1){



mutex_lock();
x = x+1;

mutex_unlock();




}

}
```

Include del mutex.
A su vez, poner `#include` <mutex.h>

```
#ifndef _MUTEX_H_
#define _MUTEX_H_

void mutex_init(void);
void mutex_lock(void);
void mutex_unlock(void);
```

```
#endif
```

Ejercicio 3

```
/*PROGRAMA A*/
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <sys/shm.h>
int main(){

int direcMemComp;
void *ptr;
void *aux;

char *PATH = "/home/marisol/TRABAJOSSO/Hola.txt";

//Solicita la dirección de memoria compartida
direcMemComp = shm_open("ejercicio3", O_CREAT | O_RDWR, 0666);
if (direcMemComp == -1) {
    perror("shm_open failed");
    exit(EXIT_FAILURE);
}
if (ftruncate(direcMemComp, 4096) == -1) {
    perror("ftruncate failed");
    exit(EXIT_FAILURE);
}

//Configura el tamaño de la memoria compartida

//Mapea la memoria compartida en el espacio de dirección del proceso S

ptr = mmap(0,4096,PROT_READ | PROT_WRITE, MAP_SHARED,direcMemComp,0);

if (ptr == MAP_FAILED){
printf("Map failed \n");
exit(-1);
}

aux = ptr; //Guardo la dirección inicial de la memoria compartida
```

```c
int direc = open(PATH, O_RDONLY);

printf("%i\n\r",direc);


if (direc != -1) { // SI direc == -1, entonces hubo un error
char buffer [4096];
ssize_t bytesLeidos;


bytesLeidos = read(direc,buffer,9); //Leemos 20 bytes


printf("Bytes leidos %zd\n",bytesLeidos);
if (bytesLeidos != -1){ //SI no hubo error, escribimos el contenido de la
lectura en la memoria compartida

sprintf(ptr,"%s", buffer); //Coloco EL CONTENIDO del archivo en la memoria
compartida


int cl = close(direc);

if (cl == -1) {
printf("ERROR EN CERRAR EL ARCHIVO");
}else{
printf("EL archivo se abrió y se cerró con éxito");}


}else{
printf("ERROR AL LEER BYTES");

}

}else{
printf("ERROR AL ABRIR EL ARCHIVO");

}

}
```

```c
/*Programa b*/
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
```

```c
#include <stdlib.h>


int main(){
void *ptr;
int direccionMemComp = shm_open("ejercicio3",O_RDONLY,0666);

if (direccionMemComp != -1) {
ptr = mmap(0,4096,PROT_READ,MAP_SHARED,direccionMemComp,0);

if (ptr == MAP_FAILED){
printf("Map failed\n");
exit(-1);



 }

 printf("%s", ptr); //Imprime el contenido de la direccion de memoria
compartida desde ptr



 if(shm_unlink("ejercicio3") == -1){
  //Remueve la porción de memoria compartida creada.
 printf("ERROR AL ELIMINAR LA MEMORIA COMPARTIDA");



 }


 }

 }
```

Función / Comando	Qué hace	Ejemplo breve
`open` (`man 2 open`)	Abre o crea un archivo y devuelve un **descriptor de archivo** (un número entero).	`int fd = open("archivo.txt", O_RDONLY);`
`read` (`man 2 read`)	Lee datos de un descriptor de archivo y los coloca en un buffer de usuario.	`read(fd, buffer, 100);`
`close` (`man 2 close`)	Cierra un descriptor de archivo y libera recursos.	`close(fd);`
`shm_overview` (`man 3 shm_overview`)	**Página de manual** que explica cómo funcionan las llamadas de memoria compartida (`shm_*`).	No es una función, es un resumen teóri
`shm_open` (`man 7 shm_open`)	Crea o abre un objeto de memoria compartida POSIX y devuelve un descriptor.	`int fd = shm_open("/mi_memoria", O_CREAT
`shm_unlink` (`man 3 shm_unlink`)	Elimina (desvincula) el objeto de memoria compartida, borrando su nombre en el sistema.	`shm_unlink("/mi_memoria");`

Ejercicio 4

```
#include "titlescreen.h"
#include "playerImage.h"
#include "enemy.h"
#include "boss.h"
#include "gameover.h"
#include "shoot.h"

extern unsigned char tecla_actual;
typedef unsigned short u16;
#define RGB(r, g, b) (r | (g << 5) | (b << 10))
// #define REG_DISPCNT *(u16 *)0x4000000
#define extern videoBuffer
#define MODE3 3
#define BG2_ENABLE (1 << 10)
#define WHITE RGB(31, 31, 31)
#define BLACK RGB(0, 0, 0)

/*
#define BUTTON_A        (1<<0)
#define BUTTON_B        (1<<1)
#define BUTTON_SELECT   (1<<2)
#define BUTTON_START    (1<<3)
```

```
#define BUTTON_RIGHT    (1<<4)
#define BUTTON_LEFT     (1<<5)
#define BUTTON_UP       (1<<6)
#define BUTTON_DOWN     (1<<7)
#define BUTTON_R        (1<<8)
#define BUTTON_L        (1<<9)
#define KEY_DOWN_NOW(key)  (~(BUTTONS) & key)
*/
//#define BUTTONS *(volatile unsigned int *)0x4000130

#define BUTTON_A    0x24
#define BUTTON_Q        0x10
#define BUTTON_B    0x25
#define BUTTON_SELECT   0x03
#define BUTTON_START    0x2c
#define BUTTON_RIGHT    0x1f
#define BUTTON_LEFT 0x1e
#define BUTTON_UP   'w'
#define BUTTON_DOWN     's'
#define BUTTON_R    '1'
#define BUTTON_L    '2'
#define KEY_DOWN_NOW(key)  (tecla_actual == key)

//variable definitions
#define playerspeed 2
#define enemyspeed 1
#define fastXSpeed 3
#define fastYSpeed 2


void setPixel(int x, int y, u16 color);
void drawRect(int x, int y, int width, int height, u16 color);
void drawHollowRect(int x, int y, int width, int height, u16 color);
void drawImage3(int x, int y, int width, int height, const u16* image);
void delay_galaga();
void waitForVBlank();

//helpers
void initialize();
void drawEnemies();
void endGame();
int collision(u16 enemyX, u16 enemyY, u16 enemyWidth, u16 enemyHeight, u16
playerX, u16 playerY);

//objects
struct Players {
    volatile u16 playerX;
    volatile u16 playerY;
};
struct Enemy {
```

```
    volatile u16 enemyX;
    volatile u16 enemyY;
};
struct FastEnemy {
    volatile u16 fastX;
    volatile u16 fastY;
};

int shoots[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
int curr_shot = 0,  arreglo[3]; //Declaro los pid como variables globales.
#define N_SHOOTS 10
void inicio(void), controlJ (void), vidasPuntaje(void), galaga(void);


void inicio(){





resume(create(galaga,1024,20,"Proceso 1",0));
resume(create(vidasPuntaje,1024,20,"Proceso 2",0));
resume(create(controlJ,1024,20,"Proceso 3",0));


}




void controlJ(){
arreglo[2] = getpid();;

int mensaje;

while(1){
mensaje = receive();


if (mensaje){
endGame();
kill(arreglo[0]);
kill(arreglo[1]);

exit();


}
```

```
}

}


void vidasPuntaje(){
int vidas = 3, puntaje = 0, mensaje;
char buf[32];


arreglo[1] = getpid();

  sprintf(buf, "Presione Q para terminar el juego :)");
  print_text_on_vga(30, 240, buf);

    sprintf(buf, "Vidas: %d",vidas);
    print_text_on_vga(30, 250, buf);

    sprintf(buf, "Puntaje: %d", puntaje);
    print_text_on_vga(30, 260, buf);

    while (1) {
    mensaje = receive();
    sprintf(buf, "Recibido: %d", mensaje);
  print_text_on_vga(30, 280, buf);



        if (mensaje == 0) { //Hubo un SHOT de un enemigo difícil. Debo
sumar puntaje.

      puntaje = puntaje + 20;


    }else {
    if (mensaje == 1) { //Hubo un SHOT. Debo sumar puntaje.

    puntaje = puntaje + 10;


    }

    else {
    if (mensaje == 2){
    vidas--;


    if (vidas == 0) {//Game over. Mando un mensaje a P3
    send (arreglo[2],1);
    break;
```

```
    }
    }
    }



}
     // Actualiza la visualización en pantalla VGA
        sprintf(buf, "Vidas: %d",vidas);
        print_text_on_vga(30, 250, buf);

        sprintf(buf, "Puntaje: %d", puntaje);
        print_text_on_vga(30, 260, buf);
    }


    sprintf(buf, "   GAME OVER   ");
print_text_on_vga(30, 250, buf);

        sprintf(buf, "Puntaje Final: %d", puntaje);
        print_text_on_vga(30, 260, buf);


}









void galaga(void) { //Este módulo dibuja los enemigos, el jugador y
chequea los SHOTS
    //easy enemy wave set setup
  arreglo[0] = getpid();
    struct Enemy easyEnemies[9];
    for (int a = 0; a < 9; a++) {
        easyEnemies[a].enemyX = (28*a);
        easyEnemies[a].enemyY = 0;
    }
    easyEnemies[1].enemyX = 240;
    easyEnemies[4].enemyX = 240;
    easyEnemies[8].enemyX = 240;
    //difficult enemies setup
    struct Enemy hardEnemies[9];
    for (int a = 0; a < 9; a++) {
```

```
        hardEnemies[a].enemyX = (28*a);
        hardEnemies[a].enemyY = 160;
    }
    hardEnemies[3].enemyX = 240;
    hardEnemies[6].enemyX = 240;
    //player setup
    struct Players player;
    player.playerX = 120;
    player.playerY = 136;
    //fast enemy "boss" setup
    struct FastEnemy fast;
    fast.fastX = 0;
    fast.fastY = 30;

    // REG_DISPCNT = MODE3 | BG2_ENABLE;
    //initalize title screen
    print_text_on_vga(10, 20, "GALAGA ");
    drawImage3(0, 0, 240, 160, titlescreen);
    while(1) { //COmienza el juego
        if (KEY_DOWN_NOW(BUTTON_START)) {
            break;
        }
    }
    //start black screen for drawing
    for (int i = 0; i < 240; i++) { //Cambia los píxeles a negro para
"dibujar"
        for (int j = 0; j < 160; j++) {
            setPixel(i, j, BLACK);
        }
    }
    while(1) {
        //go back to title screen if select button is pressed

        if (KEY_DOWN_NOW(BUTTON_Q)) { //Si el jugador presiona la letra Q
        send(arreglo[2],1); //El jugador quiere salir del juego

        }



        //player shots
        if (KEY_DOWN_NOW(BUTTON_A)) { //DIsparos del jugador
            if (shoots[curr_shot] == 0) {
                shoots[curr_shot] = 136*240 + player.playerX+9; /* 24
widht player */
                curr_shot++;
                if (curr_shot >= N_SHOOTS)
                    curr_shot = 0;
            };
        }
        //player movement input. Movimiento del jugador
```

```
        if (KEY_DOWN_NOW(BUTTON_LEFT) && (player.playerX > 0)) {
            player.playerX -= playerspeed;
        }
        if (KEY_DOWN_NOW(BUTTON_RIGHT) && (player.playerX <= 216)) {
            player.playerX += playerspeed;
        }
        if (KEY_DOWN_NOW(BUTTON_UP) && (player.playerY > 25)) {
            player.playerY -= playerspeed;
        }
        if (KEY_DOWN_NOW(BUTTON_DOWN) && (player.playerY <= 136)) {
            player.playerY += playerspeed;
        }
        waitForVBlank();
        sleepms(50);
        //draw player.
        drawImage3(player.playerX, player.playerY, 24, 24, playerImage);
        drawHollowRect(player.playerX - 1, player.playerY - 1, 26, 26,
BLACK);
        drawHollowRect(player.playerX - 2, player.playerY - 2, 28, 28,
BLACK);
        //draw easy enemies with downward movement
        for (int a = 0; a < 9; a++) {
            easyEnemies[a].enemyY += enemyspeed;
            drawImage3(easyEnemies[a].enemyX, easyEnemies[a].enemyY, 20,
20, enemy);
            if (collision(easyEnemies[a].enemyX, easyEnemies[a].enemyY,
20, 20, player.playerX, player.playerY)) {
                send(arreglo[1],2); //PIERDE UNA VIDA
                  drawRect(easyEnemies[a].enemyX, easyEnemies[a].enemyY,
20, 20, BLACK);
                 easyEnemies[a].enemyY = 0; //Movemos el enemigo hacia
arriba, así no resta vidas indefinidamente





            }
            if (easyEnemies[a].enemyY >= 160) {
                easyEnemies[a].enemyY = 0;
            }
        }

        //draw shots
        for (int i = 0; i < N_SHOOTS; i++) {
            if (shoots[i] != 0) {
                drawRect((shoots[i] % 240), (shoots[i] / 240)+4, 5, 5,
BLACK);
                drawImage3((shoots[i] % 240), (shoots[i] / 240), 5, 5,
shoot);
```

```
                shoots[i] = shoots[i]-240*4;
                if (shoots[i] <=0)   shoots[i]=0;
            }

            // check hits of shoots
            for (int j = 0; j < 9; j++) {
                if (shoots[i] != 0 && collision(easyEnemies[j].enemyX,
easyEnemies[j].enemyY, 15, 15, shoots[i] % 240, shoots[i] / 240)) {

                    drawRect(easyEnemies[j].enemyX, easyEnemies[j].enemyY,
20, 20, BLACK);
                    drawRect((shoots[i] % 240), (shoots[i] / 240)+4, 5, 5,
BLACK);
                    easyEnemies[j].enemyY = 0;
                    shoots[i] = 0;
                    send(arreglo[1],1); //Envío un mensaje al proceso dos

                }
            }
        }

        //draw hard enemies
        for (int a = 0; a < 9; a++) {
            hardEnemies[a].enemyY += enemyspeed;
            drawImage3(hardEnemies[a].enemyX, hardEnemies[a].enemyY, 20,
20, enemy);
            if (collision(hardEnemies[a].enemyX, hardEnemies[a].enemyY,
15, 15, player.playerX, player.playerY)) {

                send(arreglo[1],2); //PIERDE UNA VIDA
                  drawRect(hardEnemies[a].enemyX, hardEnemies[a].enemyY,
20, 20, BLACK);
                 hardEnemies[a].enemyY = 0; //Movemos el enemigo hacia
arriba, así no resta vidas indefinidamente


            }
            if (hardEnemies[a].enemyY >= 228) {
                hardEnemies[a].enemyY = 0;
            }
            if ((hardEnemies[a].enemyY >= 200) && (easyEnemies[a].enemyY
<=45)) {
                hardEnemies[a].enemyY = 160;
            }
            //space enemies apart
            if ((hardEnemies[a].enemyY >= 200) && (easyEnemies[a].enemyY
<=45)) {
                hardEnemies[a].enemyY = 160;
            }
            if ((easyEnemies[a].enemyY >= 120) && (hardEnemies[a].enemyY
```

```
>=170)) {
                hardEnemies[a].enemyY = 160;
            }
        }

        //SHOTS DE ENEMIGOS DIFÍCILES

        for (int i = 0; i < N_SHOOTS; i++) {
            if (shoots[i] != 0) {
                drawRect((shoots[i] % 240), (shoots[i] / 240)+4, 5, 5,
BLACK);
                drawImage3((shoots[i] % 240), (shoots[i] / 240), 5, 5,
shoot);
                shoots[i] = shoots[i]-240*4;
                if (shoots[i] <=0)   shoots[i]=0;
            }

            // check hits of shoots
            for (int j = 0; j < 9; j++) {
                if (shoots[i] != 0 && collision(hardEnemies[j].enemyX,
hardEnemies[j].enemyY, 15, 15, shoots[i] % 240, shoots[i] / 240)) {


                    drawRect(hardEnemies[j].enemyX, hardEnemies[j].enemyY,
20, 20, BLACK);
                    drawRect((shoots[i] % 240), (shoots[i] / 240)+4, 5, 5,
BLACK);
                    hardEnemies[j].enemyY = 0;
                    shoots[i] = 0;
                    send(arreglo[1],2); //Mando mensaje

                }
            }
        }




        //draw fast enemy
        drawImage3(fast.fastX, fast.fastY, 15, 15, boss);
        drawHollowRect(fast.fastX - 1, fast.fastY - 1, 17, 17, BLACK);
        drawHollowRect(fast.fastX - 2, fast.fastY - 2, 19, 19, BLACK);
        if(collision(fast.fastX, fast.fastY, 15, 15, player.playerX,
player.playerY)) {
            send(arreglo[2],1); //Avisamos al proceso 3 que debe terminar
(Aunque no estemos usando este enemigo)
        }
    //RAFA fast.fastX += fastXSpeed;
    //RAFA  fast.fastY += fastYSpeed;
        if (fast.fastX >= 240) {
```

```
            fast.fastX = 0;
        }
        if (fast.fastY >= 200) {
            fast.fastY = player.playerY - 20;
        }
    }
}


int collision(u16 enemyX, u16 enemyY, u16 enemyWidth, u16 enemyHeight, u16
playerX, u16 playerY) {
    //check if bottom right corner of enemy is within player
    if (((enemyX + enemyHeight) > playerX) && ( (enemyY + enemyHeight)
        > playerY ) &&  ((enemyX + enemyWidth) < (playerX + 24))
        && ((enemyY + enemyWidth) < (playerY + 24))) {
        return 1;
    }
    //check bottom left corner of enemy
    if ( (enemyX < (playerX + 24))
        && (enemyX > playerX)
        && ((enemyY + enemyHeight) > playerY)
        && ((enemyY + enemyHeight) < (playerY + 24))
        ) {
        return 1;

    }
    //check top left corner
    if ( (enemyX < (playerX + 24))
        && (enemyX > playerX)
        && (enemyY > playerY)
        && (enemyY < (playerY + 24))
        ) {
        return 1;

    }
    //check top right corner
    if ( ((enemyX + enemyHeight) < (playerX + 24))
        && ((enemyX + enemyHeight) > playerX)
        && (enemyY > playerY)
        && (enemyY < (playerY + 24))
        ) {
        return 1;

    }
    return 0;
}

void endGame() {
    //start Game Over State
    drawImage3(0, 0, 240, 160, gameover);
```

```
    drawHollowRect(0, 0, 240, 160, WHITE);
}
```